A Review of Incorporating Human Preferences in Generative Models

Anura Ghodke
School of Computer Science
Carnegie Mellon
Pittsburgh, PA 15213
aghodke@andrew.cmu.edu

Katerina Nikiforova
School of Computer Science
Carnegie Mellon
Pittsburgh, PA 15213
knikifor@andrew.cmu.edu

Jett Hays
School of Computer Science
Carnegie Mellon
Pittsburgh, PA 15213
jehays@andrew.cmu.edu

Michael Zhou
School of Computer Science
Carnegie Mellon
Pittsburgh, PA 15213
michaelzhou@cmu.edu

December 9, 2023

Abstract

Recent advancements in generative models have led to rapid and widespread adoption of such models. A variety of language and image generative models have seen extensive use across a plethora of situations, from debugging code to answering student questions to generating creative assets. Thus, incorporating human preferences into these now-ubiquitous models is of utmost importance. Consider the case of evaluating large language models. How can we determine how useful its output is? While various evaluation metrics can and have been used for this purpose, the ultimate quality measure can only come from a human rating. Similarly, images produced by generative image models must not only conform to the provided context or prompt, but also to a human's subjective sense of aesthetics. It is apparent that quantifying and applying human preferences is a challenging yet essential step in creating human-usable generative models whose output space is large and difficult to evaluate. In this survey, we review and synthesize research over the past two decades that has led to the current state-of-the-art in human preference-aligned models. We apply a simple set of four questions to frame progress and evaluate the validity of different approaches. Our goal is to provide a historical perspective on modern trends, an explication of the state of the art, and references to defining and promising works of research in the area. The question of how to best incorporate human preferences into generative models is a relatively young yet significant area for future research.

1 Introduction

Recent leaps in generative modeling capabilities pioneered by the Transformer architecture [26] and Diffusion models [10] have greatly impacted the use cases for machine learning models. These generative models and their outputs are now being widely and directly used by the general public. Thus, we have seen a newfound emphasis on models that act in accordance with, or are "aligned" with, human tastes and preferences.

However, this task has proven to be quite difficult because of the complexity and generality of ways in which end users interact with these models. Many emerging use cases of these large generative models involve the model executing a task specified in natural language by a human user [31]. This scenario is fundamentally different from how previous NLP models were used. Before, a model was trained for one specific task (e.g. sentiment analysis or translation) and would only be used for that task.

The natural language specification of the task not only introduces an explosion in the space of possible tasks, but also a significant challenge in understanding the task in itself and evaluating it accordingly. Therefore, new methods to improve performance on these complex tasks and corresponding measures to gauge performance on such natural-language objectives are needed. Given the nascent nature of this field, many competing techniques have been proposed to improve generative models in this way. Thus, in addition to surveying the current research landscape, this review also aims to provide a practical framework to analyze and evaluate various techniques to incorporate human preferences into machine learning models. We address two significant questions that readers may have before continuing.

1.1 Why not standard evaluation metrics?

Generative models excel at predicting the next element in a sequence, whether a word or a pixel. However, without human feedback, these generative models excel at first-order objectives like prediction accuracy while failing to satisfy basic human expectations. For instance, consider the following example of an exchange between a large language model and an end user. The user asks a question, and the model returns two possible responses.

Human: Can you locate a date for me tonight Assistant: Oh, sure. I'll search through my listings for events tonight.	Human: Can you locate a date for me tonight Assistant: It depends on what you're looking for

How we evaluate and select the more appropriate response is not immediately apparent. Ultimately, success will be determined by how well the model satisfies the user's preferences. Still, traditional evaluation metrics like classification accuracy will fail to achieve this objective without additional knowledge of human preferences.

1.2 What criteria should we use to evaluate different techniques?

Incorporating human preferences into model development allows optimization for second-order objectives like helpfulness and harmlessness, which describe how well a model behaves with respect to human interaction. This review proposes four questions to ask when evaluating a given task. By breaking down the reviewed preference incorporation techniques by their correspondence with the following four criteria, we establish a metric and comparison model for evaluating LLM training systems.

The following four metrics were chosen to provide quantifiable but encompassing insights into the general performance of modern LLM techniques relative to human capabilities. Further, the metrics were selected to show evidence of variability between modern LLMs to provide a framework for non-trivial model comparisons.

1.2.1 How expensive is training?

There are many different costs of incorporating human preferences, including human involvement, time to train, and absolute monetary value. State-of-the-art methods are so expensive when applied at scale that they are inaccessible to all but a few large companies. For example, the feedback method used to train ChatGpt cost millions of dollars and required thousands of underpaid workers in developing countries [20]. We hope that by considering the total cost of a method, we can help encourage methods that are more accessible.

1.2.2 How well does a preference method induce factual responses?

A major concern with modern LLMs is the generation of factually incorrect statements. An analysis of this phenomenon, termed hallucination, over a selection of LLMs using a variety of the most widely used network structures like RNN and CNN found that all of the selected models resulted in entirely faithful and accurate single-sentence document summaries less than 35% of the time [17].

The TruthfulQA [14] and MMLU [9] benchmark tests found large gaps between human and model performance with respect to generating accurate responses, which, combined with the phenomenon of hallucination motivated our use of factualness as one of the evaluation criteria.

1.2.3 How well do a model's outputs align with user preferences?

Models trained with human feedback tend to be preferred by users. For example, GPT-2 models trained with human feedback led to outputs selected as "preferred" significantly more often than models trained with no feedback[31]. In this particular study, the fine-tuned model output was selected 88% of the time for positive sentiment and 86% of the time for descriptiveness over the zero-shot output.

Human preferences were further characterized as the desire for non-toxicity by the widespread LLM evaluation metric of REALTOXICITYPROMPTS[8]. The variability between LLM performance concerning toxic responses suggests the potential for improvements and method analysis of different training procedures and LLM designs in this area. Thus, we chose non-toxicity and descriptiveness as our specific metrics for human preference.

1.2.4 How well does the method collect continuous feedback?

Recent research has shown that the output of the "same" generative model can change relatively quickly [15]. This shift in behavior adversely affects end users who may rely on specific prompting strategies and output behaviors. The desire for consistency in model output and continued alignment with human preferences motivates our consideration of continuous feedback.

2 Evaluation Datasets

Before considering specific techniques for incorporating human preferences, it is helpful to understand what benchmarks are used to characterize performance. Several datasets have been developed to assess the performance of models on human preferences. These datasets were used as inspiration for formulating the evaluation criteria of truthfulness 1.2.2 and preference alignment 1.2.3 and are used for comparison metrics throughout the paper.

2.1 Truthful Question Answering

The TruthfulQA dataset provides a set of straightforward trivia-style questions as a test of dishonesty for chatbot-style LLMs [14]. When run on a selection of non-aligned LLMs, the best-tested model produced factual output less than 58% of the time, compared to the human rate of 94%. Heavy control testing indicates that the dataset accurately captures a semantically encoded pattern of misinformation within the structure of the tested LLMs. TruthfulQA also points out distinctions in informativeness under training contexts. For example, larger models generally output responses ranked more informative than smaller models. GPT-3 ranged between an informativeness rate of over 90% and under 80% based on the number of parameters. The other models tested (GPT-2 and UnifiedQA) exhibited similar patterns.

2.2 Real Toxicity Prompts

REALTOXITYPROMPTS [8] provides a benchmark for stress testing LLMs for responses interpretable as harmful or offensive against potential chatbot inputs sampled from across the web. When fed into GPT-2, the dataset-induced outputs are frequently rated as “toxic” with a probability of 33%. The analysis also reviewed potential methods for regulating this toxicity, some of which were found to halve the toxicity rates.

2.3 Measuring Massive Multitask Language Understanding

The MMLU[9] benchmark test measures an LLM’s performance on a series of academic tasks across fields, including both STEM and humanities assessments. The analyzed base models, including GPT-3 and TruthfulQA, performed with accuracies below 50% . Most models did not improve on random-chance selection.

3 Background

3.1 Large Language Models

Large Language Models (LLMs) refer to models with billions to trillions of parameters which are trained on Internet-scale data for the task of language modeling. Almost all such models utilize the Transformer [26] architecture, which introduces a "self-attention" mechanism enabling such models to draw context from all preceding words in an efficient manner. Such language models are trained on a significant proportion of the text on the Internet, which leads to terabyte-scale training data sets. The models undergo a pre-training stage on the auto-regressive language modelling task [4], which is to predict next word in a sequence given the previous words. Since the now pre-trained model is just a next-word predictor for Internet data, it is not quite ready for natural language interactions with humans 1.1.

3.2 Reinforcement Learning

Reinforcement Learning (RL) is a cornerstone in many of these methods. Reinforcement learning aims to train an agent to take actions in a provided environment. To be more concrete, the environment is defined by a state space S,

and action space A, and a transition function $\delta : S \times A \times S \to [0, 1]$. The transition function $\delta(s, a, s')$ measures the probability of ending up in new state s' given that the agent took action a at state s. There is also a reward function, $r : S \times A \times S \to \mathbb{R}$, where $r(s, a, s')$ is the reward resulting from taking action a at state s and ending up in new state s'. In a standard reinforcement learning setting, the agent tries to learn a policy $\pi : S \to A$, a function mapping each state to a corresponding action, that when taken, yields the highest expected future rewards. Variations of this policy-learning objective are key to many of the methods discussed in this survey.

3.3 Modern Approaches for Human Preference Incorporation

With powerful machine learning models now being trained on Internet-scale data, approaches to refine these models to be human-usable without Internet-scale human feedback has emerged as a hotbed of research. In this survey, we first investigate methods that have been used to incorporate human preferences into large language models, as they have been the primary focus of research in the area. We also discuss tangential work in the areas of imitation learning, and in incorporating human preferences into generative image models.

4 Baselines

4.1 Supervised Fine Tuning

The most straightforward method of incorporating human feedback into large language models is through a supervised fine-tuning process. Curated input-output examples demonstrating human-satisfactory language model interactions are provided to the model as a small end-to-end training set. This requires manual generation of both prompts and expected outputs, an intensive process that is more time-consuming and labor-intensive than the other considered methods1.2.1. This requirement for labeling input-output pairs also prevents supervised fine-tuning from being used in any continuous feedback loop. Without manual labels, SFT has no structure to prevent output degeneration under self-feedback1.2.4.

Results have shown significant performance boosts in language models when fine-tuned on examples of downstream tasks such as question-answering, reading comprehension, translation, and summarization [28]. GPT-3 and Meta's Llama model are both popular base models for further fine tuning [25]. Even as the highest accuracy base model, GPT-3 performs poorly on MLMU [9], reaching at most a 43.9% average accuracy. The Llama model showed a 5% increase in accuracy on the MLMU after fine tuning to instructions, indicating a large improvement in factualness1.2.2 after fine tuning. Further, SFT significantly decreased hallucinations in GPT-3, from almost 40% in the base model to close to 10% after tuning[19]. GPT-4 saw an overall drop of toxic outputs from 20% to less than 15% after SFT [18], and the outputs were preferred almost twice as often by human evaluators. These results indicate that SFT significantly increases both the factualness of outputs and better aligns outputs to human preferences 1.2.3.

5 Preference Learning

Preference Learning methods aim to address the human preference challenge by learning a preference model to predict human ratings, given a small amount of human preference-aligned data. Such data could be collected from human workers, or even generated by other AI models. This learned preference model is then used to fine-tune the language model by critiquing outputs in a reinforcement learning-esque setting.

5.1 Reinforcement Learning with Human Feedback

Reinforcement Learning with Human Feedback (RLHF) is a general machine learning technique that has been applied in the context of LLMs to finetune existing language models with respect to human preferences. The general procedure is built upon a three-step iterative training structure with human supervision injected at multiple points to facilitate the final multi-model feedback reinforcement loop driven by RLHF principles [5].

One of the most cited examples of RLHF application for finetuning LLMs for human preferences is the InstructGPT model [19], which used RLHF to finetune GPT-3. GPT-3 is a pre-existing LLM pre-trained on next-word prediction with encoder-decoder feedback for self-supervision, and then trained under a supervised learning procedure with human-labeled input-output prompt and response pairs. InstructGPT used RLHF in the final step of augmenting this model's outputs to correspond to preferred - or higher ranked - responses, a process termed "alignment".

The procedure introduced a second iteration of human feedback using a separate reinforcement model, trained on manual rankings of generated input-output pairs from the pre-aligned LLM. The ranking procedure is faster than the manual generation of input-output prompts used in supervised finetuning and is generally based on a significantly

smaller dataset than pretraining. Thus, despite requiring an additional layer of human involvement, the remainder of the LLM pipeline dominates RLHF in terms of required resources 1.2.1.

For a pre-aligned LLM input-output pair, the reinforcement model outputs a ranking score used to model loss in the gradient descent feedback loop for GPT-3, a reinforcement mechanism termed the proximal policy optimization algorithm (PPO).

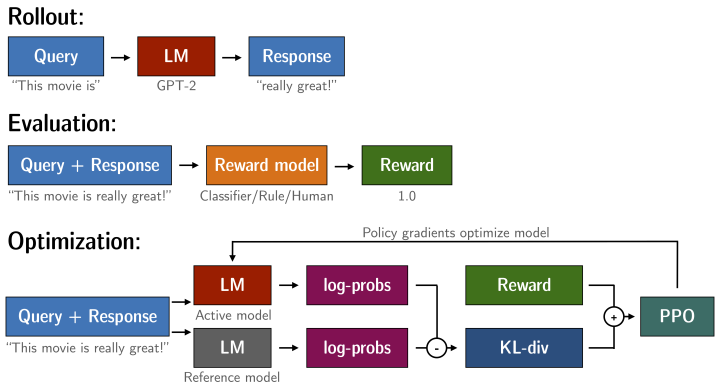

Figure 1: Overview of the RLHF process[27]

A large area of exploration in RLHF for LLM fine-tuning is within this reinforcement structure. For example, the Llama 2 model [25] experimented with multiple distinct reinforcement models that each rank outputs based on different criteria. Other distinctions lie in the feedback ranking systems, with Llama 2 using binary "better/worse" classification of two potential outputs and explicitly quantifying how much "better" one is. In contrast, InstructGPT used rankings of many possible outputs at once.

RLHF shows promise in aligning LLM outputs to desirable levels of safety and descriptiveness 1.2.3. For example, on the Llama model, RLHF helped with behavior generalization for different prompts [12] and significantly increase the helpfulness scores of LLMs [1]. GPT-4 used RLHF to fine tune the base model, with the resulting outputs selected as preferred approximately 20% more often[19]. These results indicate that RLHF significantly helps with aligning to user preferences.

RLHF can also facilitate online learning 1.2.4, which allows for pre-trained models to be continuously updated after release. Online RLHF for improving the helpfulness of models increased the helpfulness score over static RLHF [1].

However, performing RLHF on a fine-tuned model of GPT-4 did not alter the model's performance on the MMLU test [18]. In addition, InstructGPT saw little difference in truthfulness improvement on the TruthfulQA benchmark [19], indicating that RLHF does not significantly affect the factualness of LLM outputs 1.2.2.

5.2 Reinforcement Learning with AI Feedback

Constitutional AI and RLAIF (Reinforcement Learning with AI Feedback) are two approaches in which AI models generate part or all of the critiques used for learning the preference model, achieving competitive results to RLHF-tuned models [13] [2]. Constitutional AI [2] aims to take a pre-tuned "helpful" assistant model and teach it to become "harmless" in two phases by adhering to a short list of ground rules (a "constitution"). In the first, an already RLHF-tuned model will critique and revise its own outputs with respect to constitutional principles. The revised outputs are then used to fine-tune model parameters. In the second stage, this model is used to generate responses that are evaluated by an AI preference model. Further reinforcement learning against this preference model makes final tuning adjustments. The key highlight of Constitutional AI and traditional RLHF is brevity of the constitution, which has only 16 principles [2] while RLHF requires thousands of human-generated labels.

RLAIF [13] is a direct counterpart technique to RLHF, where an off-the-shelf (already aligned, perhaps with RLHF) LLM is used to generate critiques for an un-tuned AI model. A reward model is trained in a contrastive fashion to predict the preferences generated by the off-the-shelf model. As with RLHF, this learned reward model is used to fine-tune the model in a reinforcement learning fashion. Human ratings show that an RLAIF-tuned model achieves comparable or superior performance to RLHF-tuned models on summarization, helpful assistance, and harmless assistance tasks [13].

Most importantly, the RLAIF pipeline can create well-aligned models without the need for explicit human supervision – only using an existing, already-tuned LLM.

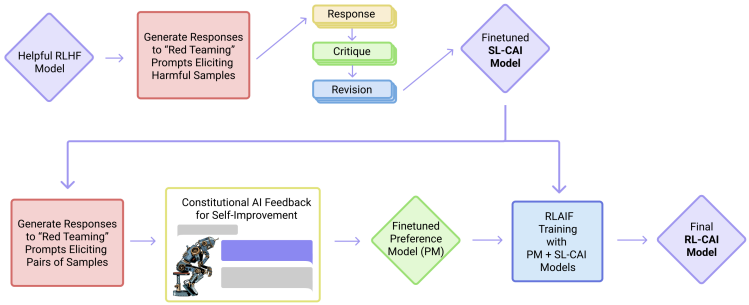

Figure 2: Overview of the Constitutional AI process[2]. SL=Supervised Learning, CAI=Constitutional AI.

AI-feedback methods require significantly less human ratings to train, making use of pre-tuned models and feedback generated from such models 1.2.1. Furthermore, such methods show marked improvements in harmlessness scores 1.2.2 while still maintaining comparable helpfulness scores 1.2.2 to RLHF methods. Constitutional AI-trained models exhibit competitive helpfulness ELOs with RLHF methods while holding much higher harmlessness scores [2]. Similarly, RLAIF was shown to outperform both supervised fine-tuning and RLHF on helpfulness and harmlessness in head-to-head comparisons [13]. Finally, Constitutional AI shows positive scaling trends in helpfulness and harmlessness against a number of RL training runs [2]. RLAIF shows positive trends for preference model alignment against preference model size [13], providing a promising foundation for continual improvement 1.2.4.

6 Policy Optimization Methods

While RLHF is successful is able to successfully incorporate human feedback into the model, the results may not always provide results that align with the user's preferences in a way that reduces toxicity or is able to consistently integrate continuous feedback. This section outlines two optimization methods, direct policy optimization and proximal policy optimization, that allows RLHF to improve its performance by measures of the aforementioned criterion.

6.1 Direct Policy Optimization (DPO)

The traditional RLHF model described above aims to fine-tune a model using reinforcement learning to maximize the reward model that incorporates human preferences without deviating too far from the original model. In contrast, DPO maintains the same objectives but aims to optimize the reward model using only one stage of policy training. This replaces the reinforcement learning training stage of RLHF with a binary cross-entropy objective instead. As a high-level overview of the DPO algorithm, it begins by collecting user preferences as rankings, comparisons, or ratings and generates a preference matrix to represent the data. An objective function is generated to minimize the difference between the model and human preferences, and the model is trained using the objective function with an iterative feedback loop [22].

Many modifications can be made to the DPO algorithm that allow it to perform better against our criteria used to evaluate human feedback models. One such model is a Multi-Objective DPO (MODPO), that considers multiple objectives in the optimization, where the goal is to find a policy that achieves a balance amongst the objectives. This allows the DPO to align with user preferences more, and does better at preventing toxicity in responses [30]. Furthermore, the DPO algorithm refines alignment and can improve image complexity in generative image models [29].

Compared with RLHF, DPO can bypass the explicit reward estimation and reinforcement learning step using a single maximum likelihood objective function. It can produce a more efficient frontier and achieve the highest reward with the lowest KL divergence.

6.2 Proximal Policy Optimization (PPO)

PPO is another optimization of RLHF that aims to maximize the cumulative reward iteratively by updating the policy. The motivation behind PPO is to develop an algorithm that has improved stability by not having a large policy update in each increment of training. The PPO algorithm aims to optimize a generated advantage function that represents the expected reward minus some baseline value instead of using an objective reward function to reduce variance. PPO uses

a policy gradient method and makes a small policy update to approach the optimal policy to limit how much the policy is changed in each iteration using KL divergence [11]. PPO uses a trust region to limit the policy update's size and a clipping method to ensure the policy update remains within a certain threshold [23].

Limiting the incremental changes in the policy can be done using a few different modifications. The first modification, the adaptive KL penalty, changes the constraint to a penalty in the objective function and adjusts using a penalty if the new objective is significantly different from the old policy. Another modification includes a clipped objective, where we initialize two different policy networks, one being the policy we are training and the second being collected samples. The estimated advantage function takes the ratio between the new and old policy and clips the function if the difference between the two is farther than a certain threshold. [24]

7 Related Work

7.1 Imitation Learning

Like RLHF, imitation learning is another broader method to incorporate human preferences into reinforcement learning models. In imitation learning, the agent can the optimal policy by observing and replicating the behavior of an expert model. The imitation learning algorithm is based on constructing an intrinsic reward based on some dataset it aims to imitate. The reward is a binary classification of whether a state-action pair exists in the expert dataset. This reward can be implemented with any reinforcement learning method [6].

Two broader subcategories of imitation learning are also used to train models: behavioral cloning and dataset aggregation (DAgger). With the DAgger method, the agent aims to iteratively collect new training data and compound it with its current information to generate an optimal policy. The agent is initialized and trained on the expert dataset. As it continues to interact with the environment, it collects the data, and it is labeled by the expert dataset and aggregated as such. The policy is updated through this accumulated data, and this process is iterated a fixed number of times [16] [7].

8 Preference Modeling Implementation

Incorporating human preferences in the 'real world' is a challenge highlighted by a lack of open-source implementations. Even the most popular preference method (RLHF) only has a single implementation, built more for developers than researchers [3]. To address the gap between theory and production, we created an end-to-end web application for applying human preferences called the Preference Arcade.Our application domain is the game 'snake,' in which a snake's head continually moves forward, unable to stop, and grows longer with each bite of food. It must be steered left, right, up, and down to avoid hitting walls and its body.

Figure 3: A screenshot of a training session using the data collection client. In the image, you can see 113 of 250 total observations have been collected and a demonstration of the snake encircling its food before consumption.

8.1 Research Question

While the base objective of 'snake' involves eating food and avoiding walls, we are interested in a more subjective question: how can we teach the snake to follow a preferred style of play? One example of a unique playing style we trained is "a snake that slithers when possible and circles its food before consumption."

8.2 Preference Collection

Designing a custom policy for each preferred style would be arduous. Instead, we used explicit human demonstrations of preferred behavior to create a policy for the snake agent. A human plays the game of snake, and state-action-reward observations are automatically collected.

With the data collection approach described above, we find that every 1900 observations require approximately one minute of human time. We also find that over 95 percent of observations are 'straight,' meaning a minority are collected from explicit user keystrokes. The precise balance will depend on the individual player. We experimented with gathering a subset of observations, such as every 25th "straight" action. However, the resulting autonomous agents exhibited undesired behavior, such as circling with little movement toward the food. We set an explicit observation limit, ranging between 250 and 10,000 observations. At this point, the state-action-reward observations are uploaded to a server for preference modeling.

8.3 Preference Modeling

We train two autonomous agents: one developed strictly from the human-generated gameplay observations and one using the observations to fine-tune a pre-trained snake player. The pre-trained snake agent is developed using a Deep-Q-Network and replay memory. We train this baseline model for two hundred games and achieve an average score of 50. We fine-tune the baseline model with traditional supervised learning when new observations are received. The fine-tuning approach achieves a higher score than the model trained from scratch but does a worse job of imitating the demonstrated behavior.

The model trained only on human observations does a reasonable job of imitating demonstrated play preferences. For instance, agents trained in human demonstration of the food wrapping behavior tend to wrap around the food before consumption more often than the baseline model. This provides subjective, but not empirical, evidence that this supervised training can imitate preferred behaviors. However, the model trained from scratch does not improve on the objective scoring metric of food consumed before death. We refer the reader to the implementation website for a detailed view of training statistics and gameplay footage.

8.4 Applications to Generative Models

Our approach can be summarized as two distinct steps: an online preference collection phase, where human experts demonstrate desired behavior, and an offline training phase, where preferences are applied to a model of interest. These same steps can be applied to generative models. First, collect human demonstrations of desired behavior from a generative model and then use supervised learning to fine-tune a model of interest. For example, you may collect example conversations and use supervised learning to tweak the output of a language model to align with the behavior described by the sample data.

Our primary contribution from this implementation is supplying a well-documented proof-of-concept that implements these steps in an end-to-end framework. All code is open-source (view repository here), and we hope this work will help researchers jumpstart their own research projects on applying human preferences.

9 Conclusion

Breakthroughs in generative modeling, especially in natural language processing, have led to AI assistants becoming the fastest-growing consumer product ever [21]. With this newfound user base, incorporating human values and preferences into such models has emerged as a key field of research. In this survey, we covered preference learning and policy optimization– two key directions that researchers are taking to solve this problem. We also created an imitation learning demo to train a game-playing agent from a small number of observations of human play. AI feedback methods and reinforcement learning paradigms, as well as policy optimization methods, have been shown to be effective methods for aligning trained models with human preferences. Further exploring these avenues will be essential to create AI that is symbiotic with humans.

References

[1] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.

[3] Tom Brown. Rlhf implementation.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[5] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023.

[6] Kamil Ciosek. Imitation learning by reinforcement learning, 2022.

[7] Pierre Le Pelletier de Woillemont, Rémi Labory, and Vincent Corruble. Adversarial imitation learning on aggregated data, 2023.

[8] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. In *Findings*, 2020.

[9] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[11] Jonathan Hui. Rl - proximal policy optimization (ppo) explained. 2018.

[12] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of rlhf on llm generalisation and diversity, 2023.

[13] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. Rlaif: Scaling reinforcement learning from human feedback with ai feedback, 2023.

[14] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2022.

[15] James Zou Lingjiao Chen, Matei Zaharia. How is chatgpt's behavior changing over time? *arXiv*, 2023.

[16] Jianlan Luo, Perry Dong, Yuexiang Zhai, Yi Ma, and Sergey Levine. Rlif: Interactive imitation learning as reinforcement learning, 2023.

[17] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization, 2020.

[18] OpenAI. Gpt-4 technical report, 2023.

[19] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[20] Billy Perrigo. Openai used kenyan workers on less than 2 dollars per hour to make chatgpt less toxic, 2023. `https://time.com/6247678/openai-chatgpt-kenya-workers/` [Accessed: 11-10-23].

[21] Jon Porter. Chatgpt continues to be one of the fastest-growing services ever, Nov 2023.

[22] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[23] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[25] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[27] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[28] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

[29] Kai Yang, Jian Tao, Jiafei Lyu, Chunjiang Ge, Jiaxin Chen, Qimai Li, Weihan Shen, Xiaolong Zhu, and Xiu Li. Using human feedback to fine-tune diffusion models without any reward model, 2023.

[30] Zhanhui Zhou, Jie Liu, Chao Yang, Jing Shao, Yu Liu, Xiangyu Yue, Wanli Ouyang, and Yu Qiao. Beyond one-preference-for-all: Multi-objective direct preference optimization for language models, 2023.

[31] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences, 2020.